# Artificial Intelligence Assignment 6


### By Himanshu Sardana


## Question 1

Create a dataset (.csv file) having following features- experience of the candidate, written score, interview score and salary. Based on three input features, HR decide the salary of the selected candidates. Using this data, KNN model build a for HR department that can help them decide salaries of the candidates. Predict the salaries for the following candidates, by executing the model (for different values of K): (a) 5 Yrs experience, 8 written test score, 10 interview score (b) 8 Yrs experience, 7 written test score, 6 interview score

```
import random
import csv
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor


def generate_dataset():
    features = ["Experience", "Interview Score", "Written Score"]
    output = "Salary"

    data = []
    for _ in range(1000):
        experience = random.randint(1, 20)
        interview_score = random.randint(1, 10)
        written_score = random.randint(1, 10)
        salary = (experience * 2000) + (interview_score * 10) +
(written_score * 5) + random.randint(-5000, 5000)
        data.append([experience, interview_score, written_score, salary])

    with open("dataset.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(features + [output])
        writer.writerows(data)
        print("Dataset generated")

def build_knn_model():
    data = pd.read_csv("dataset.csv")
    X = data.iloc[:, :-1]
```

```
    y = data.iloc[:, -1]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    knn_model = KNeighborsRegressor(n_neighbors=5)
    knn_model.fit(X_train, y_train)

    accuracy = knn_model.score(X_test, y_test)
    print(f"Model Accuracy: {accuracy * 100:.2f}%")

# build_knn_model()

def predict_salary(experience, interview_score, written_score):
    data = pd.read_csv("dataset.csv")
    X = data.iloc[:, :-1]  # Features
    y = data.iloc[:, -1]   # Target variable (Salary)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    knn_model = KNeighborsRegressor(n_neighbors=5)
    knn_model.fit(X_train, y_train)

    prediction = knn_model.predict([[experience, interview_score,
written_score]])
    return prediction[0]

print(predict_salary(5, 8, 10))
```

## Question 2

**Create a dataset (.csv file) having following features- Graduations percentage, experience of the candidate, written score, interview score and selection. Selection feature is binary in nature and contains the status of the candidate. Also store at least 25 records in this dataset. Using this data, build a Bayesian learning model for HR department that can help them to decide whether the candidate will be selected or not. Take 80% data as training data and remaining a testing data randomly. Using the built model, predict the status for the following unseen data: (a) 90 %, 5 Yrs experience, 8 written test score, 10 interview score (b) 75%, 8 Yrs experience, 7 written test score, 6 interview score Also calculate the possible classification metrics for the above cases and save these values in the .CSV file.**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from random import randint
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.naive_bayes import GaussianNB
import numpy as np
import csv

def generate_data(num_records):
    data = []
    for _ in range(num_records):
        graduation_percentage = randint(50, 100)
        experience = randint(0, 20)
        written_score = randint(0, 10)
        interview_score = randint(0, 10)
        selection = 1 if (graduation_percentage > 70 and experience > 2 and
written_score > 5 and interview_score > 5) else 0
        data.append([graduation_percentage, experience, written_score,
interview_score, selection])
    return data

# generated_data = generate_data(25)
# columns = ['Graduation Percentage', 'Experience', 'Written Score',
'Interview Score', 'Selection']
# df = pd.DataFrame(generated_data, columns=columns)
# df.to_csv('candidate_data.csv', index=False)

# Load the dataset
df = pd.read_csv('candidate_data.csv')
X = df[['Graduation Percentage', 'Experience', 'Written Score', 'Interview
Score']]
y = df['Selection']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
predictions = gnb.predict(X_test)

# Predicting for unseen data
unseen_data = np.array([[90, 5, 8, 10], [75, 8, 7, 6]])
predictions_unseen = gnb.predict(unseen_data)
print(f"Predictions for unseen data: {predictions_unseen}")

accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
```

```
print(f"Recall: {recall}")
print(f"F1 Score: {f1}")
```

## Question 3

**For the IRIS dataset, design a decision tree classifier. Take different percentage of training data and then observe effect on the accuracy and other quality parameters. Also note the effect of other decision tree parameters (like max depth, min_sample_spit etc.) on the performance of the model. Note: Take criterion as entropy**

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import numpy as np

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

clf_default = DecisionTreeClassifier(random_state=69)
clf_default.fit(X_train, y_train)
y_pred_default = clf_default.predict(X_test)

accuracy_default = accuracy_score(y_test, y_pred_default)
precision_default = precision_score(y_test, y_pred_default,
average='weighted')
recall_default = recall_score(y_test, y_pred_default, average='weighted')
f1_default = f1_score(y_test, y_pred_default, average='weighted')
print(f"Default Decision Tree Classifier - Accuracy: {accuracy_default},
Precision: {precision_default}, Recall: {recall_default}, F1 Score:
{f1_default}")

percentages = [0.1, 0.2, 0.3, 0.4, 0.5]
results = []

for percentage in percentages:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1-
percentage, random_state=42)

    clf = DecisionTreeClassifier(random_state=69)
```

```
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    results.append((percentage, accuracy, precision, recall, f1))

# Print results
print("Results for different training percentages:")
print("Percentage | Accuracy | Precision | Recall | F1 Score")
for result in results:
    print(f"{result[0]:<11} | {result[1]:<8.4f} | {result[2]:<8.4f} |
{result[3]:<6.4f} | {result[4]:<6.4f}")

depths = [1, 2, 3, 4, 5]
results_depth = []
for depth in depths:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    clf = DecisionTreeClassifier(max_depth=depth, random_state=69)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    results_depth.append((depth, accuracy, precision, recall, f1))

print("Results for different max depths:")
print("Max Depth | Accuracy | Precision | Recall | F1 Score")
for result in results_depth:
    print(f"{result[0]:<9} | {result[1]:<8.4f} | {result[2]:<8.4f} |
{result[3]:<6.4f} | {result[4]:<6.4f}")
```

## Question 4

**By taking Classified Data as input, compare the performance of KNN, Bayesian Classifier and Decision Tree model. In this comparison, you can take different possible parameters of particular model. Save these output in a .csv file.**

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
import numpy as np

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=69)

clf_default = DecisionTreeClassifier(random_state=69)
clf_default.fit(X_train, y_train)
y_pred_default = clf_default.predict(X_test)
accuracy_default = accuracy_score(y_test, y_pred_default)
precision_default = precision_score(y_test, y_pred_default,
average='weighted')
recall_default = recall_score(y_test, y_pred_default, average='weighted')
f1_default = f1_score(y_test, y_pred_default, average='weighted')

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn, average='weighted')
recall_knn = recall_score(y_test, y_pred_knn, average='weighted')
f1_knn = f1_score(y_test, y_pred_knn, average='weighted')

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)
accuracy_gnb = accuracy_score(y_test, y_pred_gnb)
precision_gnb = precision_score(y_test, y_pred_gnb, average='weighted')
recall_gnb = recall_score(y_test, y_pred_gnb, average='weighted')
f1_gnb = f1_score(y_test, y_pred_gnb, average='weighted')

results = {
    'Model': ['Decision Tree', 'KNN', 'GaussianNB'],
    'Accuracy': [accuracy_default, accuracy_knn, accuracy_gnb],
    'Precision': [precision_default, precision_knn, precision_gnb],
    'Recall': [recall_default, recall_knn, recall_gnb],
    'F1 Score': [f1_default, f1_knn, f1_gnb]
}
```

```
results_df = pd.DataFrame(results)
results_df.to_csv('model_comparison_results.csv', index=False)
```